Wireless Networks — Course Project

Rate-Limited Sender Increase in MsQuic CUBIC

Implementing and evaluating
`draft-ietf-ccwg-ratelimited-increase-11`

Abhiram Suresh

November 2026

Code. MsQuic branch `rate-limited-increase` (commits `dd0428a62`, `19538311e`, `7c6397cf3` and the upstream bug fix `1ca13e9d7`), on top of upstream `464c873c0`. Patch files are in `patches/`; experiment scripts in `experiments/`; everything needed to rebuild this document and its figures is in `report/`.

Abstract

A *rate-limited* sender is one that is not using its whole congestion window: a video player between segments, an RPC client, a game, a heartbeat. CUBIC grows the congestion window (*cwnd*) with time and on every acknowledgement, so a rate-limited sender can keep growing *cwnd* although the network never carried that much of its data. When the application later has a lot to send, it releases the whole inflated window at once and overflows the bottleneck queue. The IETF draft *Increase of the Congestion Window when the Sender Is Rate-Limited* bounds *cwnd* by the largest flight used since the last window reduction, *maxFS*. MsQuic's CUBIC had such a bound with two gaps: *maxFS* was never reset, and the bound was $2 \cdot maxFS$ in congestion avoidance too. We close both gaps (34 lines of code, four new unit tests) and show with a deterministic ablation that each of our five changes matters. Evaluating the patch on an emulated bottleneck (420 runs, 11 settings) we also found an *upstream bug*: an unsigned underflow in CUBIC's Reno-friendly accumulator that makes *cwnd* grow by one packet per ACK in congestion avoidance; we fix it as well. With the bug fixed in both builds, after 28 s of rate-limited sending our change cuts *cwnd* at resumption from 670 to 166 KB and the data lost after resuming from 1094 to 32 KB, at no cost in transfer time; in shorter periods it still cuts losses by 34–63 %. The patch needs the bug fix: without it, a competing TCP flow can stall. On a 50 Mbit/s path with a shallow queue the conservative window costs 23 % in completion time of the following 8 MB transfer.

Contents

1 Introduction: why this matters **2**

2 Background **2**
2.1 Congestion window, flight size and slow start . . . 2
2.2 Why CUBIC is especially exposed . . . 3
2.3 The draft's rules . . . 3

3 What stock MsQuic does, and the two gaps **4**
3.1 A worked example of both gaps . . . 4

4 Implementation **5**
4.1 A design decision: *ssthresh* is left alone . . . 6
4.2 Instrumentation . . . 6

5 Per-change examples: the scenario harness and ablation **7**
5.1 Scenario A — resuming after a long rate-limited period (C1, C2, C3) . . . 7
5.2 Scenario B — why *ssthresh* is not lowered (design decision) . . . 8
5.3 Scenario C — a spurious loss is undone (C5) . . . 8
5.4 Scenario D — a heartbeat sender (C4 and the design decision) . . . 8
5.5 Scenario E — ECN (C1 and C2 on the ECN path) . . . 9

6 An upstream bug found by the evaluation **10**

7 Network evaluation **12**
7.1 Testbed and workload . . . 12
7.2 Sweep: stock vs. patched across settings . . . 13
7.3 Four builds: separating our change from the bug . . . 14

8 Unit tests **16**

9 Discussion and limitations **16**

10 Conclusion **17**

A Audit of the earlier plots in `experiments/` **18**

B Reproducing the results **18**

1 Introduction: why this matters

Most real Internet traffic is not a long bulk transfer. It comes in bursts separated by quiet periods, and between the bursts the sender has less to send than its congestion window would allow. Examples:

- **Adaptive video.** A player downloads a few seconds of video, then waits for the buffer to drain, then downloads again.
- **Request/response protocols** (HTTP/3, RPC). The sender is limited by how fast requests arrive, not by the network.
- **Real-time and interactive traffic** (games, telemetry, heartbeats). The sender emits a few small packets per round trip.

The congestion window exists to answer one question: *how much data can this path safely carry right now?* The only evidence the sender has is the data it actually sent and got acknowledged. If the sender keeps only 29 KB in flight but lets *cwnd* grow to 400 KB, the extra 371 KB is a guess nobody checked. The problem shows up when the application suddenly has a lot to send (the next video chunk, a large response): the sender is allowed to release the whole window in one round trip. That burst lands in the bottleneck queue, the queue overflows, packets of *other* flows are dropped and their latency jumps.

Figure 1 shows the problem on a real network stack (Section 7). After a loss, the application sends only about 25 KB per round trip for 28 s. Stock MsQuic's CUBIC keeps growing *cwnd* until it hits a limit computed from a flight it last used half a minute earlier: 670 KB, 27 times what is in flight. When the application then has 8 MB to send, the whole window leaves at once and about 1.1 MB is lost. The patched sender keeps *cwnd* at what it has used and loses 24 KB in this run (median over 10 runs: 32 KB against 1094 KB).

Contributions.

1. An analysis of where MsQuic's CUBIC deviates from the draft (two gaps, Section 3).
2. A patch that closes both gaps, made of five small changes C1–C5 (Section 4), plus four new and two updated unit tests.
3. A deterministic scenario harness and an ablation study that shows, for *each* change separately, the situation in which it matters and what goes wrong without it (Section 5).
4. The discovery and fix of an upstream bug in MsQuic's CUBIC, the underflow of the Reno-friendly accumulator (Section 6).
5. A testbed evaluation with 420 runs: a sweep over the length of the rate-limited period, RTT, bottleneck rate and a competing TCP flow, and a four-build experiment that separates the effect of our change from that of the bug (Section 7).

2 Background

2.1 Congestion window, flight size and slow start

The sender may have at most *cwnd* bytes unacknowledged. *FlightSize* is the number of bytes actually in flight. A sender is *cwnd-limited* when $\mathit{FlightSize} \approx \mathit{cwnd}$ and *rate-limited* (also called application-limited) when $\mathit{FlightSize} < \mathit{cwnd}$ because the application has nothing more to send. In *slow start* ($\mathit{cwnd} < \mathit{ssthresh}$) the window doubles every round trip; in *congestion avoidance* ($\mathit{cwnd} \geq \mathit{ssthresh}$) it grows slowly. On a loss or ECN mark the window is reduced (by CUBIC's $\beta = 0.7$) and *ssthresh* is set to the new window.

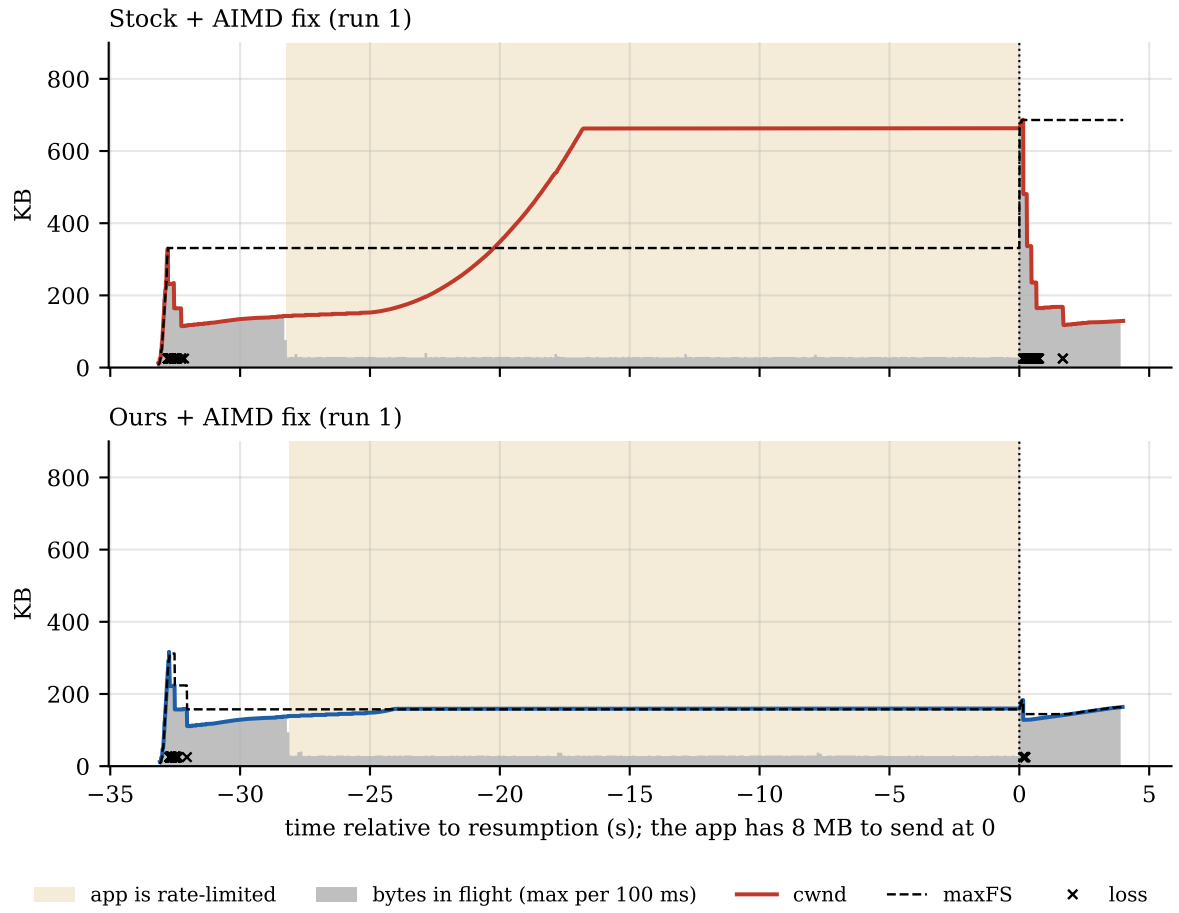


Figure 1: One run of the 28 s rate-limited setting. The AIMD bug of Section 6 is fixed in both builds, which changes nothing in this setting, so the difference is the draft's rule alone. x: loss detected.

2.2 Why CUBIC is especially exposed

CUBIC (RFC 9438) computes the window as a function of *time since the last congestion event*, $W(t) = C\,(t - K)^3 + W_{\max}$, not as a function of the number of ACKs. As long as ACKs keep arriving, the window follows this curve. A rate-limited sender still receives ACKs (for its few packets), so the window climbs the cubic curve exactly as if the path had been fully used. The longer the rate-limited period, the larger the unvalidated window. In our testbed the window grew from 150 KB to 670 KB during 28 s in which only about 25 KB was ever in flight (Figure 1).

2.3 The draft's rules

`draft-ietf-ccwg-ratelimited-increase-11` updates RFC 5681, RFC 9002 and others with a simple rule. It introduces

$$\textit{maxFS} = \text{the largest } \textit{FlightSize} \text{ since the last time } \textit{cwnd} \text{ was reduced,}$$

and requires that, while rate-limited, the window does not grow past what *maxFS* justifies:

These rules are the successor to Congestion Window Validation (RFC 7661), which instead *decays cwnd* after idle/rate-limited periods. The draft takes the simpler approach of never letting the window run ahead of the evidence in the first place.

Table 1: The rules of the draft that this project implements.

Rule	Requirement
R1 Reset	When *cwnd* is reduced for any reason, *maxFS* is reset to 0 and re-measured from the following *FlightSize*.
R2 Slow start	$cwnd \leq 2 \cdot maxFS$ (allows one doubling over what was used).
R3 Congestion avoidance	$cwnd \leq maxFS + 1 \cdot SMSS$ (allows the normal one-packet-per-RTT probe over what was used).

3 What stock MsQuic does, and the two gaps

MsQuic's CUBIC (`src/core/cubic.c`) already tracks *maxFS* in a field called `BytesInFlightMax`. It is updated on every send (`OnDataSent`) and is used at the end of `OnDataAcknowledged` to cap the window:

Stock `cubic.c`, end of `OnDataAcknowledged`

```
// Using 2 * BytesInFlightMax for the limit allows for exponential growth
// in the window when not otherwise limited.
if (Cubic->CongestionWindow > 2 * Cubic->BytesInFlightMax) {
    Cubic->CongestionWindow = 2 * Cubic->BytesInFlightMax;
}
```

The field is set to $cwnd/2$ at initialisation and on reset, and otherwise only ever grows. (The draft initialises *maxFS* to the initial window instead; we kept MsQuic's value, see Section 9.) Compared with Table 1:

Gap 1: *maxFS* is never reset (violates R1). After a loss the window is cut to 70 %, but *maxFS* keeps the pre-loss peak. The cap $2 \cdot maxFS$ is then computed from a flight size that the network just showed it *cannot* carry.

Gap 2: the congestion-avoidance cap is $2 \cdot maxFS$ (violates R3). In congestion avoidance the window may be twice the largest flight, instead of one packet more than it.

3.1 A worked example of both gaps

Take the situation of Scenario A (Section 5.1). Units are packets of 1232 B.

1. The sender is cwnd-limited and reaches $cwnd = 40$, with 40 packets in flight. So $maxFS = 40$.
2. 12 of the 40 are lost. CUBIC cuts the window to $cwnd = ssthresh = 0.7 \cdot 40 = 28$.
3. The application now sends only 4 packets per RTT for 10 seconds (200 RTTs of 50 ms).

Table 2: Window limits in the worked example after the loss. With stock MsQuic, CUBIC's time-driven growth fills all the headroom it is allowed.

	maxFS after loss	cap in CA	*cwnd* after 10 s at 4 pkt/RTT
Stock (Gap 1 + Gap 2)	40 (stale)	$2 \cdot 40 = 80$	**76.5**
Draft rules	$0 \to 28$ (reset, re-measured)	$28 + 1 = 29$	**29**

With stock MsQuic the sender ends up with a 76-packet window, *bigger than the 40 packets that just caused a loss*, although it never had more than 4 packets in flight after the loss. When the application resumes, all 76 packets go out in one round trip.

4 Implementation

The patch adds 34 lines of code and 30 lines of comments to `cubic.c` and `cubic.h` (one new field), not counting the experiment-only trace logger of Section 4.2. It consists of five changes. Figure 2 shows where each one sits in the CUBIC callbacks.

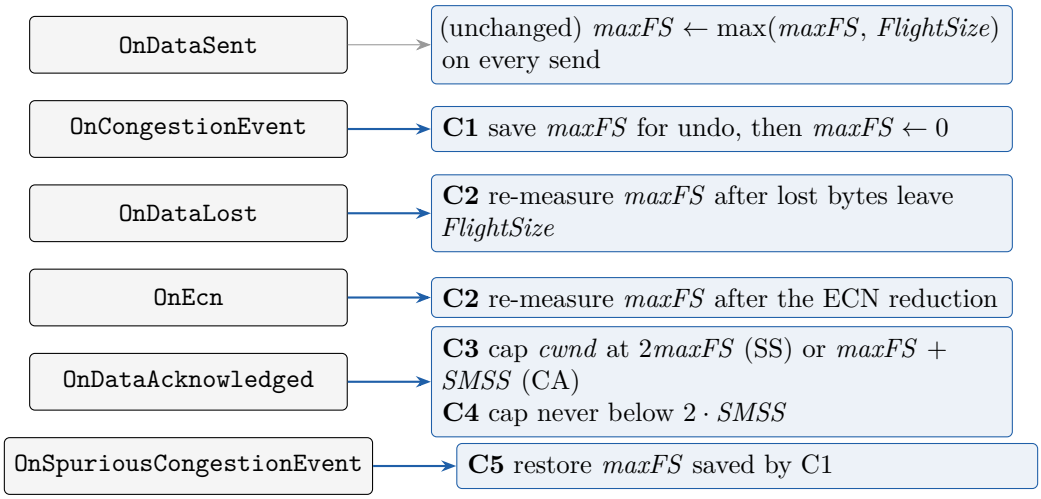


Figure 2: The CUBIC callbacks and the change made in each (`OnDataLost` and `OnEcn` both call `OnCongestionEvent`). Blue boxes are new behaviour; the grey box is existing behaviour the patch relies on.

C1 — reset *maxFS* on every congestion event (closes Gap 1)

`CubicCongestionControlOnCongestionEvent` is the single place where CUBIC reduces the window for loss, ECN and persistent congestion, so one line there implements R1 for all of them. The old value is saved first so that a spurious loss can be undone (C5).

```
        Cubic->PrevBytesInFlightMax = Cubic->BytesInFlightMax;
    ...
    // "upon a reduction of cwnd (for any reason), maxFS MUST be reset to zero"
    Cubic->BytesInFlightMax = 0;
```

C2 — re-measure *maxFS* right after the reduction

After a reset to 0, the draft says *maxFS* is measured again from the next *FlightSize*. If we waited for the next `OnDataSent`, any ACK arriving before that would see $maxFS = 0$. So both callers of the congestion event measure it immediately, at the point where *FlightSize* is already correct: in `OnDataLost` *after* the lost bytes have been subtracted (otherwise lost packets would count as "carried"), and in `OnEcn` after the reduction.

```
static void
CubicCongestionControlUpdateMaxFlightSize(_In_ QUIC_CONGESTION_CONTROL_CUBIC* Cubic)
{
    if (Cubic->BytesInFlightMax < Cubic->BytesInFlight) {
        Cubic->BytesInFlightMax = Cubic->BytesInFlight;
    }
}
```

C3 — a phase-dependent cap (closes Gap 2), and C4 — a floor

```
if (Cubic->CongestionWindow > 2 * Cubic->BytesInFlightMax) {
    Cubic->CongestionWindow = 2 * Cubic->BytesInFlightMax;
}
const uint32_t Smss = QuicPathGetDatagramPayloadSize(&Connection->Paths[0]);
const BOOLEAN InSlowStart = Cubic->CongestionWindow < Cubic->SlowStartThreshold;
uint32_t CwndLimit;
if (InSlowStart) {
    CwndLimit = 2 * Cubic->BytesInFlightMax;          // R2
} else {
    CwndLimit = Cubic->BytesInFlightMax + Smss;       // R3
}
const uint32_t MinWindow = Smss * QUIC_PERSISTENT_CONGESTION_WINDOW_PACKETS;
if (CwndLimit < MinWindow) {                          // C4
    CwndLimit = MinWindow;
}
if (Cubic->CongestionWindow > CwndLimit) {
    Cubic->CongestionWindow = CwndLimit;
}
```

C4 is not in the draft's text but is required by QUIC: RFC 9002 says the window must never go below the minimum window of 2 packets (`QUIC_PERSISTENT_CONGESTION_WINDOW_PACKETS`). Without it, a sender whose flight is a single small packet would get a window smaller than one full packet (Scenario D).

C5 — restore *maxFS* when a loss turns out to be spurious

MsQuic undoes a congestion event when the "lost" packets are acknowledged after all. It already restored *cwnd*, *ssthresh* and the CUBIC state from `Prev*` copies. Because C1 now changes *maxFS* on the congestion event, it has to be restored too, otherwise the first ACK after the undo would cap the restored window with the post-loss *maxFS* (Scenario C).

```
uint32_t PrevBytesInFlightMax;                    // cubic.h
Cubic->BytesInFlightMax = Cubic->PrevBytesInFlightMax;   // OnSpuriousCongestionEvent
```

4.1 A design decision: *ssthresh* is left alone

When C3 lowers the window below *ssthresh*, the sender is technically back in slow start, and on the next ACK the slow-start cap $2 \cdot maxFS$ applies, so *cwnd* can rise again until it reaches *ssthresh*. The window therefore oscillates, but always within $\min(ssthresh, 2 \cdot maxFS)$.

An earlier version of the patch also lowered *ssthresh* to the capped window to stop the oscillation. The ablation (Scenario B and D) showed this is harmful: *ssthresh* is the sender's memory of the last *real* congestion, and lowering it without any congestion signal makes the sender leave slow start early when the application later becomes cwnd-limited. In Scenario D, the sender sent 23 packets in the 15th RTT with *ssthresh* untouched and only 9 with it lowered (Figure 6). The draft also does not ask for it. We therefore reverted it (commit `7c6397cf3`). All network results are for the final version. In the testbed the cap pushed *cwnd* below *ssthresh*, where the rejected variant would have acted, only in 2 of 10 runs per build of the competing-TCP setting and in no run of any other setting.

4.2 Instrumentation

For the experiments, a CSV logger (`CUBIC_CWND_LOG`) records *cwnd*, *maxFS*, *FlightSize* and *ssthresh* at every send, ACK and loss. It is compiled only with `-DQUIC_CUBIC_CWND_LOG` and has no effect on normal builds. The sample client (`src/tools/sample/sample.c`) gained options to produce a controlled rate-limited workload (`-prime`, `-burst`, `-pause`, `-cycles`, `-final`) and prints the connection statistics at the end.

5 Per-change examples: the scenario harness and ablation

Network experiments show the overall effect but cannot tell which change is responsible, and they are noisy. To show precisely *where* each change matters, we drive CUBIC directly through its callbacks inside MsQuic's own unit-test harness. Every scenario is a deterministic script of sends, ACKs and losses (one round = one 50 ms RTT, $SMSS = 1232$ B, HyStart off), and is run against seven builds of `cubic.c`:

Table 3: Builds used in the ablation. Each "no_" build is the final patch with exactly one change switched off by a compile-time macro.

Build	Meaning
stock	upstream `cubic.c`
full	the final patch (C1–C5)
no_reset	without C1/C2 (*maxFS* never reset)
no_cacap	without the CA part of C3 (cap stays $2 \cdot maxFS$)
no_floor	without C4
no_restore	without C5
ssthresh_lower	the rejected design of Section 4.1

Table 4 summarises the outcome; the subsections explain each scenario. Reading the table column by column shows that every change has a scenario in which removing it, and only it, changes the result.

Table 4: Ablation summary (units: packets of 1232 B unless stated). Bold marks the build in each column that differs from full because the change under test is missing.

	A: burst on resume	C: *cwnd* on 1st ACK after undo	D: *cwnd* in heartbeat phase	D: packets sent in 15th RTT of bulk	E: *maxFS* after ECN
stock	76	29.5	25 861 B	25	40
full	29	29.5	2 464 B	23	20
no_reset	**41**	29.5	**25 861 B**	25	**40**
no_cacap	**56**	29.5	2 464 B	23	20
no_floor	29	29.5	**600 B**	22	20
no_restore	29	**22.0**	2 464 B	23	20
ssthresh_lower	29	29.5	2 464 B	**9**	20

5.1 Scenario A — resuming after a long rate-limited period (C1, C2, C3)

Setup. The worked example of Section 3.1: slow start to 40 packets, 12 lost, 200 rounds at 4 packets per round, then the application becomes cwnd-limited and sends a full window per round.

What happens. Figure 3(a) shows the window during the rate-limited period. Stock CUBIC follows its cubic curve up to 76.5 packets. Removing either half of the fix still leaves a large window:

- *no_reset* (Gap 1 only fixed in CA): *maxFS* stays 40, the CA cap is $40 + 1 = 41$.
- *no_cacap* (Gap 1 fixed, Gap 2 not): *maxFS* is correctly 28, but the cap is $2 \cdot 28 = 56$.
- *full*: *maxFS* is 28 and the cap is $28 + 1 = 29$, one packet above *ssthresh*, exactly what R3 allows.

Figure 3(b) shows the consequence: the number of packets released in the first round after resuming. The patched sender sends 29, the stock sender 76, i.e. 2.6 times more, onto a path that dropped packets at 40.

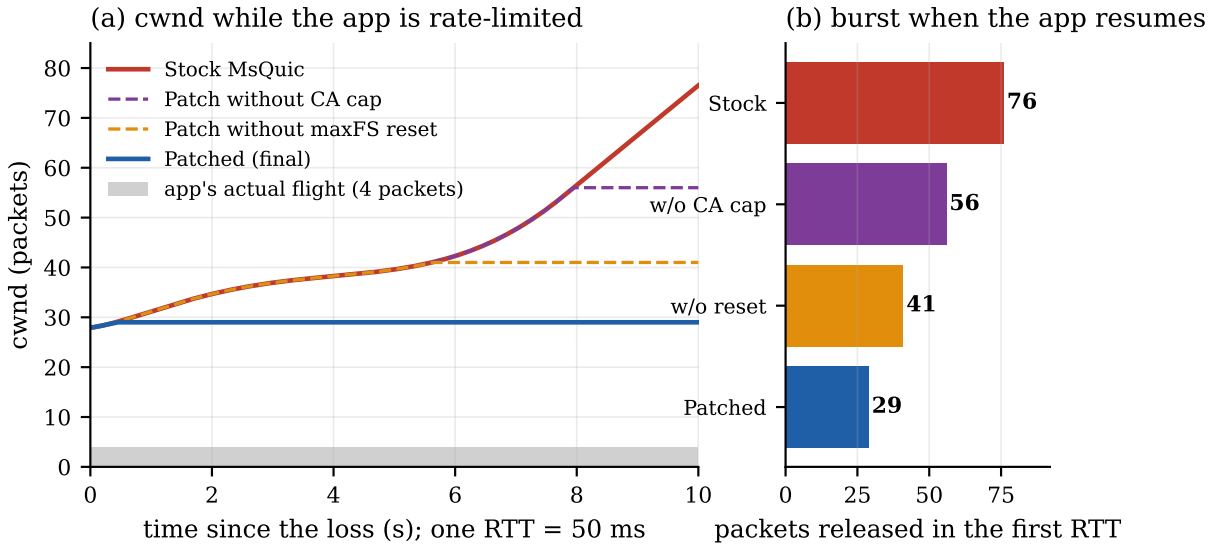


Figure 3: Scenario A. (a) *cwnd* during 200 rounds in which only 4 packets per round are sent. (b) Burst size in the first round after the application resumes. Both fixes are needed; each alone leaves a burst well above the 40 packets at which the loss occurred.

5.2 Scenario B — why *ssthresh* is not lowered (design decision)

Setup. Like A, but 20 of the 40 packets are lost, so the re-measured $maxFS = 20$ and the CA cap $20 + 1 = 21$ is *below* $ssthresh = 28$. 40 rate-limited rounds, then 15 cwnd-limited rounds.

What happens. Figure 4. With the final patch, the cap drops *cwnd* to 21, the sender is then in slow start and the $2 \cdot maxFS$ cap lets *cwnd* rise back to *ssthresh*; the window moves between 21 and 28 and never exceeds *ssthresh*. When the application resumes, the sender is already in slow start and ramps to 31 packets in 15 rounds. With *ssthresh* lowered to 21, the window is flat during the rate-limited period but the sender then grows by only one packet every two rounds, reaching 27. The stock sender is at 34–36 throughout, above the 28 that *ssthresh* says is safe.

5.3 Scenario C — a spurious loss is undone (C5)

Setup. In congestion avoidance with $cwnd \approx 29$ and a full window in flight, 30 % of the flight is declared lost; then MsQuic learns the loss was spurious and calls `OnSpuriousCongestionEvent`; then the rest of the flight is acknowledged.

What happens. Figure 5. The undo restores *cwnd* to 29.3 in every build. But C1 had set *maxFS* to the post-loss flight of 21. Without C5, the first ACK applies the CA cap $21 + 1 = 22$ and the undo is silently cancelled: the sender loses 25 % of its window for a loss that never happened. With C5, *maxFS* is restored to 36 and the window continues at 29.5, as in stock.

5.4 Scenario D — a heartbeat sender (C4 and the design decision)

Setup. After a loss at 40 packets, the application sends one 300-byte heartbeat per round for 25 rounds; one heartbeat is lost. Then the application starts a bulk transfer.

What happens. Figure 6(a): during the heartbeat phase the stock window stays at 25 861 B (21 full packets) although the flight is 300 B: when the bulk transfer starts it sends 20 packets

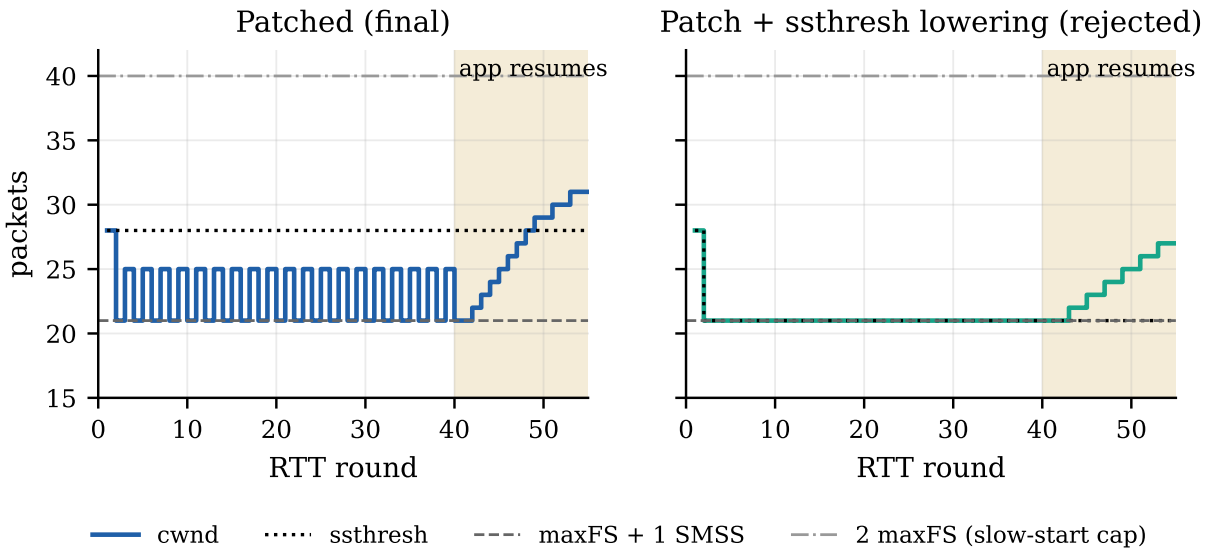


Figure 4: Scenario B. Left: the final patch. Right: the rejected variant that lowers *ssthresh*. The oscillation of the final patch stays below *ssthresh* and costs nothing; lowering *ssthresh* slows the later ramp-up.

at once. With the draft rules the cap is $maxFS + SMSS = 300 + 1232$, which C4 raises to the QUIC minimum window of $2 \cdot 1232 = 2464$ B. Without C4 the window falls to 600 B, *smaller than one full-size packet*, which RFC 9002 forbids and which costs the sender an extra round trip when it resumes.

Figure 6(b) shows the ramp-up when the bulk transfer starts. The full patch slow-starts 2, 4, 8, 16 and reaches *ssthresh* in four rounds, then grows in congestion avoidance. The rejected *ssthresh*-lowering variant has *ssthresh* at 2 packets and grows by one packet every two rounds: it sends 9 packets in the 15th round instead of 23.

5.5 Scenario E — ECN (C1 and C2 on the ECN path)

Setup. A 40-packet window is in flight; 20 packets have been acknowledged when an ECN Congestion Experienced mark arrives. 100 rate-limited rounds at 4 packets follow.

What happens. Table 5. ECN reduces the window from 60 to 42 packets in every build. The patched sender then re-measures *maxFS* from the 20 packets still in flight; the stock sender keeps 40. During the rate-limited period the stock window keeps climbing the cubic curve to 56.6 packets. The patched window never exceeds *ssthresh* (42) and ends at 37 (Figure 7). It moves between the CA cap and *ssthresh*: each time the cap drops *cwnd* below *ssthresh*, CUBIC is in slow start again, where the draft allows up to $2 \cdot maxFS$. This is the bounded oscillation of Section 4.1.

Table 5: Scenario E (packets).

	cwnd before ECN	*cwnd* after ECN	*maxFS* after ECN	*cwnd* after 100 rate-limited rounds
stock	60	42	40	56.6 (still growing)
full	60	42	20	37 (bounded by *ssthresh*)

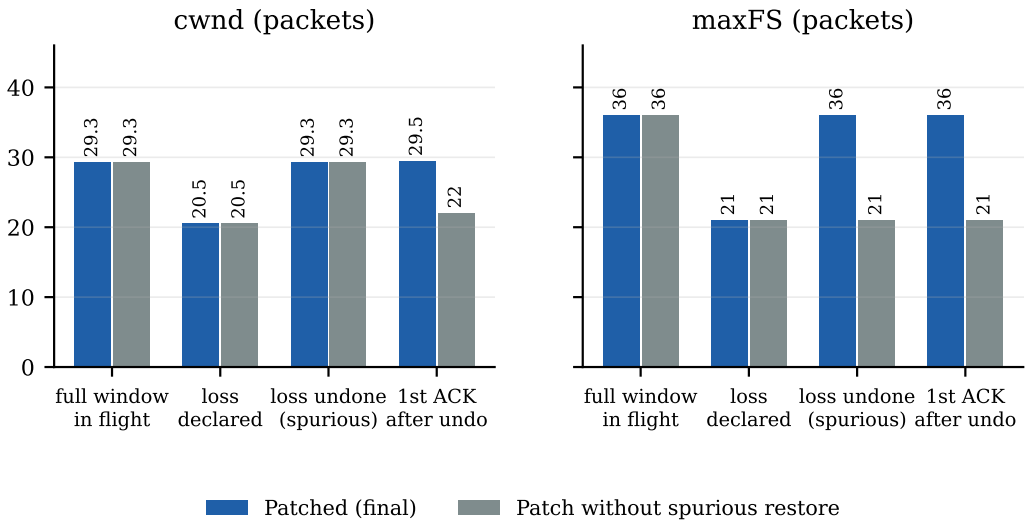


Figure 5: Scenario C. *cwnd* and *maxFS* at each step of a spurious loss and its undo. Without C5 the restored window is capped on the next ACK.

6 An upstream bug found by the evaluation

The first testbed campaign (Section 7.2) produced one result we could not explain with the draft's rules: with a competing TCP flow, the patched sender sometimes regrew its window after a loss at slow-start speed while in congestion avoidance. The cause is a bug in MsQuic's CUBIC that is independent of our patch and still present in upstream `main`.

Mechanism. CUBIC keeps a Reno-friendly estimate `AimdWindow` next to the cubic curve and uses *cwnd* = `AimdWindow` whenever it is the larger one. The estimate grows by one MSS each time an accumulator of ACKed bytes exceeds it:

```
if (Cubic>AimdAccumulator > Cubic->AimdWindow) {
    Cubic->AimdWindow += DatagramPayloadLength;
    Cubic->AimdAccumulator -= Cubic->AimdWindow;    // subtracts the GROWN window
    Cubic->AimdAccumulator -= Cubic->AimdWindow;    // subtract the window crossed
    Cubic->AimdWindow += DatagramPayloadLength;
}
```

Both fields are `uint32_t`. If the accumulator exceeds the window by less than one MSS, which is the normal case for ACKs of one or two packets, the subtraction wraps to about 4.3×10^9. From then on the condition is true on every ACK, so `AimdWindow`, and with it *cwnd*, grows by one MSS per ACK: exponential growth in congestion avoidance. Congestion events reset `AimdWindow` but not the accumulator, so the effect survives losses. MsQuic's own test `AIMD_AccumulatorTriggersWindowGrowth` chooses its ACK size "to avoid unsigned underflow in the subtraction": the underflow was known in the test, but not fixed in the code.

Evidence.

- *Harness* (scenario F: loss, 10 s rate-limited, resume, a second loss, then one ACK per packet). Stock MsQuic's accumulator wraps and *cwnd doubles every round* after the loss, from 89 KB to 2.8 MB in five rounds (Figure 8). With the fix it grows by about 1 KB per round, as CUBIC should.
- *Real traces.* The signature of the bug is a congestion-avoidance increase of exactly one SMSS on one ACK. In stock MsQuic 11–33 % of all congestion-avoidance increases have it; with the fix, 0–6 % (Figure 11, lower left).

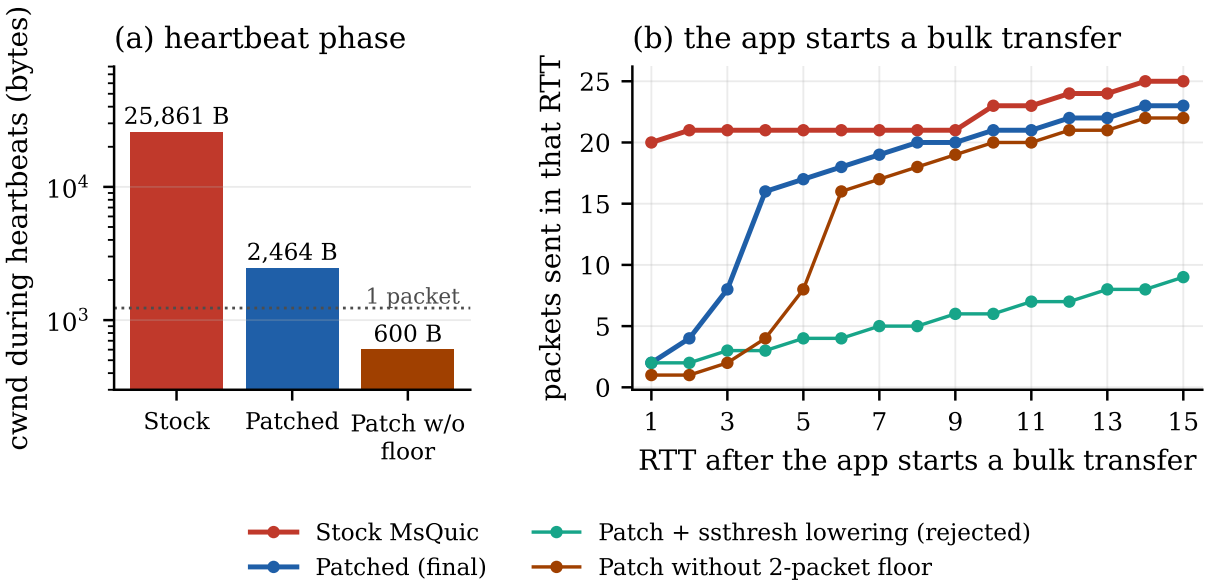


Figure 6: Scenario D. (a) Window while one 300 B heartbeat is sent per round (log scale). (b) Packets sent per round once the bulk transfer starts.

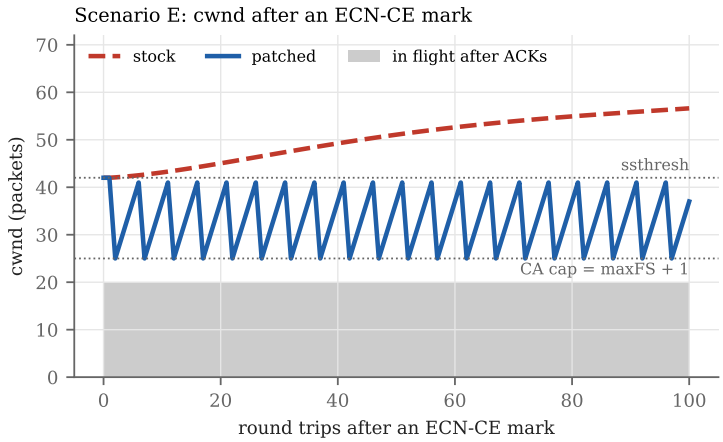


Figure 7: Scenario E. After the ECN mark, 20–24 packets stay in flight. Stock grows *cwnd* along the cubic curve; the patch keeps it between *maxFS* + 1 packet and *ssthresh*.

- *Interaction with our patch.* Our cap *maxFS* + *SMSS* does not stop this growth on a real network: every ACK lets the sender put one more packet in flight, which raises *maxFS* and so the cap. Without the fix, our patch therefore still shows the signature in 2–24 % of increases.

Fix and test. The two lines are swapped (msquic commit `1ca13e9d7`, `patches/0004`). A new test, `AIMD_AccumulatorNoUnderflow`, crosses the window by 50 bytes and fails on the old code; two tests that encoded the old order were updated. Because the bug inflates stock MsQuic's numbers, we re-ran the key settings with four builds to separate its effect from ours (Section 7.3).

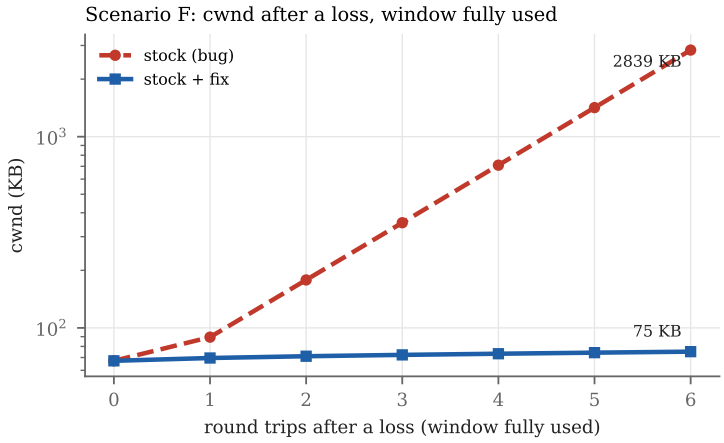


Figure 8: Scenario F, stock MsQuic with and without the accumulator fix. After a loss, with the window fully used and one ACK per packet, the wrapped accumulator makes *cwnd* double every round trip in congestion avoidance.

7 Network evaluation

7.1 Testbed and workload

Linux network namespaces form a dumbbell (Figure 9). The bottleneck is the r1–r2 link, shaped by `netem` in both directions: 10 Mbit/s by default, 20 ms delay each way (base RTT 41 ms), and a 100-packet drop-tail queue. Segmentation offloads (GSO, TSO, GRO) are turned off on every interface, so the queue limit counts real packets: with UDP GSO one queued buffer can hold many packets. A second client sends a ping every 10 ms through the same bottleneck; in one setting it also runs a long-lived TCP CUBIC flow (`iperf3`), started 3 s before the QUIC connection.

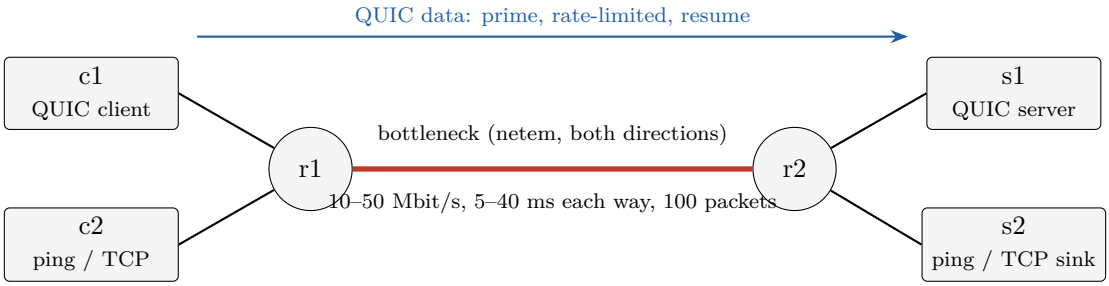


Figure 9: Dumbbell topology. The QUIC client uploads to s1; c2's ping or TCP flow shares the bottleneck queue.

One QUIC connection runs three phases: **prime** (3 MB as fast as possible, so the window grows and meets losses as in normal operation), **rate-limited** (6000 B every 10 ms, about 4.8 Mbit/s, roughly 25 KB in flight), and **resume** (8 MB as fast as *cwnd* allows). Time 0 is the start of the 8 MB transfer. We control how long the application paces itself (250–3000 bursts, nominally 2.5–30 s), but at 10 Mbit/s the first ≈5 s of that are spent draining the 3 MB prime. We therefore report the time the sender was *really* rate-limited, measured from its last cwnd-limited send (flight ≥ 90 % of *cwnd*) to resumption: 0, 0.7, 6, 17 and 28 s for the five pacing lengths, the same in every build.

Campaigns and statistics.

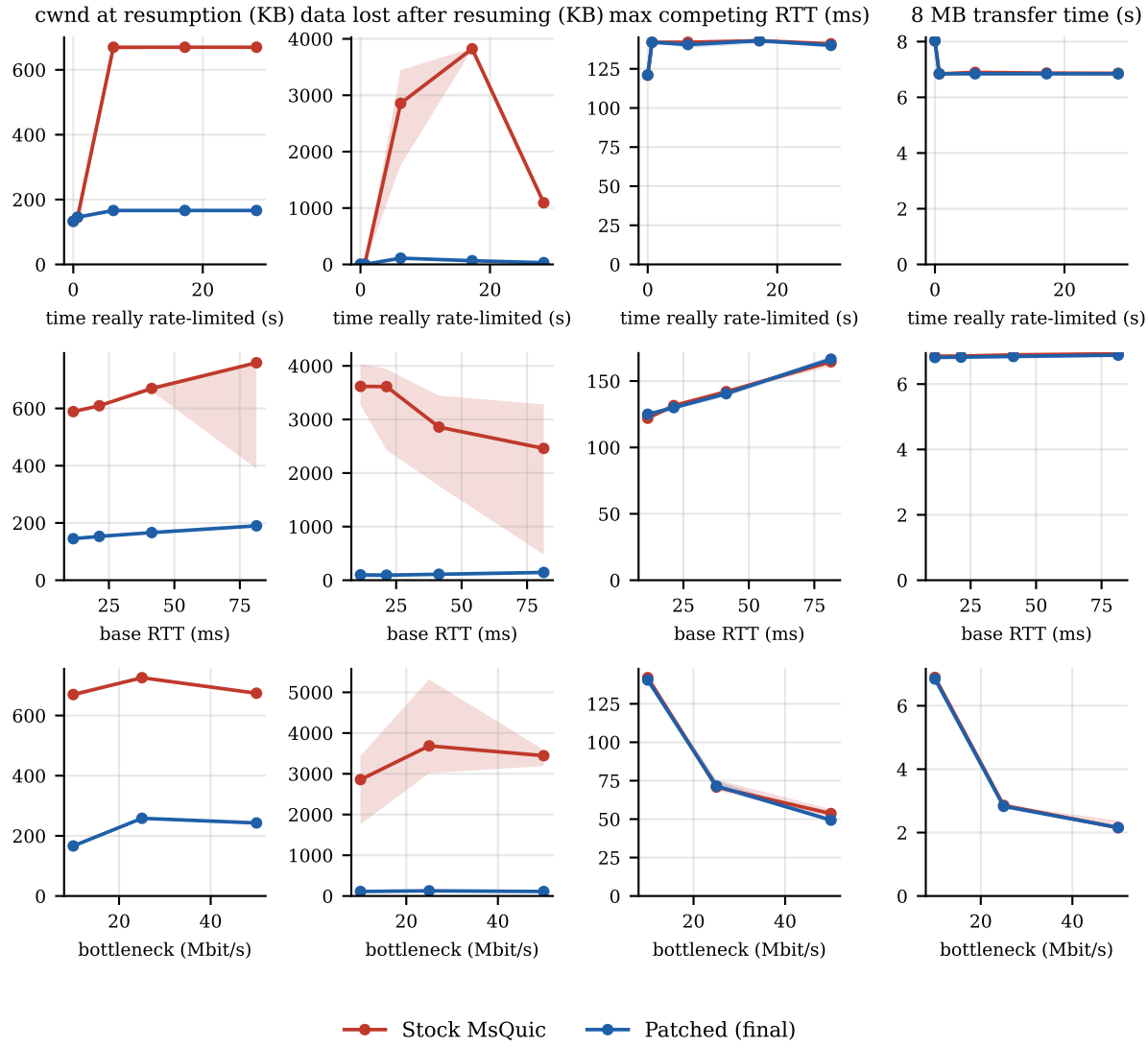


Figure 10: Sweep campaign: medians (lines) and interquartile ranges (bands) over 10 runs per point. Rows vary the length of the rate-limited phase, the RTT and the bottleneck rate; the other two stay at 6 s, 41 ms, 10 Mbit/s. The x axis of the first row is the measured time the sender was rate-limited.

Sweep (Section 7.2): 11 settings (length of the rate-limited phase, RTT, bottleneck rate, competing TCP) × stock and patched × 10 runs = 220 runs.

Four builds (Section 7.3): 5 settings × 4 builds × 10 runs = 200 runs. The builds are stock, stock + AIMD fix, ours, and ours + AIMD fix.

Runs are interleaved across settings and the build order rotates every repetition, so slow drift of the machine cannot favour one build. We report medians and quartiles, the difference of medians with a 95 % bootstrap confidence interval, and a two-sided permutation test (20 000 permutations). All 420 runs completed.

7.2 Sweep: stock vs. patched across settings

Figure 10 and Table 6 show three things.

- **The harm needs time to build up.** With 0 and 0.7 s of real rate-limited sending, CUBIC has not had time to grow the window and both builds behave the same. From 6 s on, stock resumes with 590–760 KB against 145–260 KB for the patch, at every RTT and

Table 6: Sweep campaign, median stock / **patched**. *: $p < 0.01$ (permutation test).

Point	Bottleneck, RTT, rate-limited for	*cwnd* at resume (KB)	lost after resume (KB)	max ping RTT (ms)	8 MB time (s)	n
dur_2s	10 Mbit/s, RTT 41 ms, 0.0 s	133 / **133**	1.5 / **2.2**	121 / **121**	8.0 / **8.0**	10
dur_5s	10 Mbit/s, RTT 41 ms, 0.7 s	145 / **146**	3.7 / **1.5**	142 / **142**	6.8 / **6.8**	10
dur_10s	10 Mbit/s, RTT 41 ms, 6.2 s	670 / **166***	2857 / **111***	142 / **140**	6.9 / **6.8***	10
dur_20s	10 Mbit/s, RTT 41 ms, 17 s	670 / **166***	3824 / **66***	143 / **143**	6.9 / **6.8***	10
dur_30s	10 Mbit/s, RTT 41 ms, 28 s	670 / **166***	1093 / **31***	141 / **140**	6.9 / **6.8***	10
rtt_10	10 Mbit/s, RTT 11 ms, 6.3 s	589 / **145***	3617 / **101***	122 / **125***	6.9 / **6.8***	10
rtt_20	10 Mbit/s, RTT 21 ms, 6.3 s	609 / **153***	3613 / **94***	132 / **130**	6.9 / **6.8***	10
rtt_80	10 Mbit/s, RTT 81 ms, 6.0 s	760 / **190***	2460 / **145***	164 / **166**	6.9 / **6.9***	10
bw_25	25 Mbit/s, RTT 41 ms, 9.6 s	726 / **258***	3686 / **127***	71 / **72**	2.9 / **2.8**	10
bw_50	50 Mbit/s, RTT 40 ms, 10 s	674 / **243***	3445 / **110***	54 / **49**	2.2 / **2.2**	10
compete	25 Mbit/s, RTT 41 ms, 3.3 s, +TCP	108 / **73**	57 / **43**	67 / **67**	7.3 / **7.1**	10

Table 7: Four builds, medians over 10 runs. p: our change with the bug fixed in both builds (ours + fix vs. stock + fix).

Setting	Metric	Stock	Stock + fix	Ours	Ours + fix	p
6 s rate-limited	cwnd at resumption	668	229	166	166	<0.001
	data lost after resuming	3413	82	109	30	<0.001
	8 MB transfer time	6.9	6.8	6.8	6.8	0.137
28 s rate-limited	cwnd at resumption	670	671	166	166	<0.001
	data lost after resuming	1094	1094	31	32	<0.001
	8 MB transfer time	6.9	6.9	6.8	6.8	<0.001
RTT 80 ms	cwnd at resumption	762	232	190	190	<0.001
	data lost after resuming	2800	84	121	40	<0.001
	8 MB transfer time	7.0	6.9	6.9	6.9	0.438
50 Mbit/s	cwnd at resumption	668	288	241	238	<0.001
	data lost after resuming	3002	140	105	92	<0.001
	8 MB transfer time	2.1	2.0	2.2	2.5	<0.001
+ TCP flow	cwnd at resumption	128	88	87	77	0.037
	data lost after resuming	106	47	123	21	0.060
	8 MB transfer time	7.4	8.0	4.8	7.6	0.257

bottleneck rate.

- **The harm is loss, not delay.** Stock loses 1.1–3.8 MB after resuming; the patch 31–145 KB. With a real 100-packet queue the queueing delay is capped at about 120 ms at 10 Mbit/s, which both builds reach; the maximum ping RTT therefore differs by only a few milliseconds. (In an earlier, exploratory run with GSO enabled, where the effective queue was longer, the burst raised the competing RTT from 43 ms to a median maximum of 242 ms with stock and 194 ms with the patch, and up to 368 and 213 ms in the worst runs; both builds there still had the accumulator bug.)
- **No cost in completion time** at 10–25 Mbit/s: the 8 MB transfer is as fast or slightly faster with the patch.

The competing-TCP setting was inconclusive, which led us to the bug in Section 6.

7.3 Four builds: separating our change from the bug

Table 7, Figures 11–12 and Figure 1 answer the question the bug raised.

1. **Most of stock's damage at 6 s comes from the bug.** With the fix, stock resumes with 229 instead of 668 KB and loses 82 instead of 3413 KB: the wrapped accumulator had

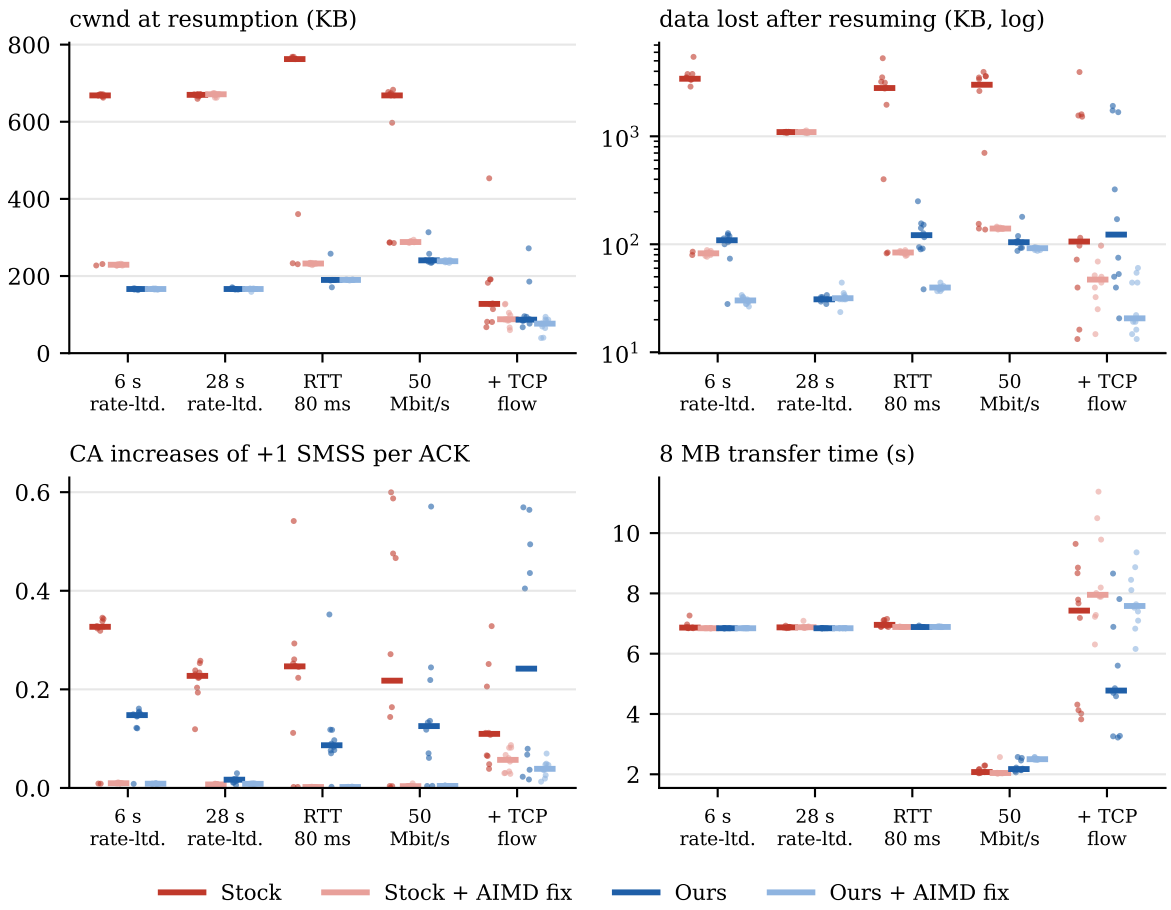


Figure 11: Four builds, 10 runs each (dots) and medians (bars). Lower left: share of congestion-avoidance increases that are exactly one SMSS on one ACK, the signature of the accumulator bug.

pushed *cwnd* up to the stale $2 \cdot maxFS$ limit in a few seconds.

2. **After long rate-limited periods the draft's rule is what matters.** At 28 s, the fix changes nothing (670 KB, 1094 KB lost): plain CUBIC reaches the stale limit on its own (Figure 1). Our change cuts this to 166 KB and 32 KB, and the 8 MB transfer is about 30 ms faster.

3. **With the bug fixed in both, our change still helps at every setting**: *cwnd* at resumption −17 to −28 % at 6 s, RTT 80 ms and 50 Mbit/s, and data lost after resuming −34 to −63 % (all $p < 0.001$).

4. **Our change needs the fix.** With a competing TCP flow, ours without the fix is the worst build for TCP (median 11.0 Mbit/s in the 2 s after resumption, worst 0.5 s 6.3 Mbit/s, three runs where TCP stalls completely for 0.5 s), because the bug's per-ACK growth passes through the moving cap. Ours + fix is the best: 15.5 and 14.7 Mbit/s, and the least QUIC data lost (21 KB vs. 47 KB for stock + fix, $p = 0.06$).

5. **A cost on high-BDP paths with a shallow queue.** At 50 Mbit/s the resumed burst overflows the 100-packet queue in all builds (first loss after 60 ms). CUBIC then cuts to 70 % of the starting window and grows slowly from there, so starting lower (238 vs. 288 KB) costs throughput: the 8 MB transfer takes 2.5 s instead of 2.0 s and the first MB 52 ms longer. This is the conservative side of the draft.

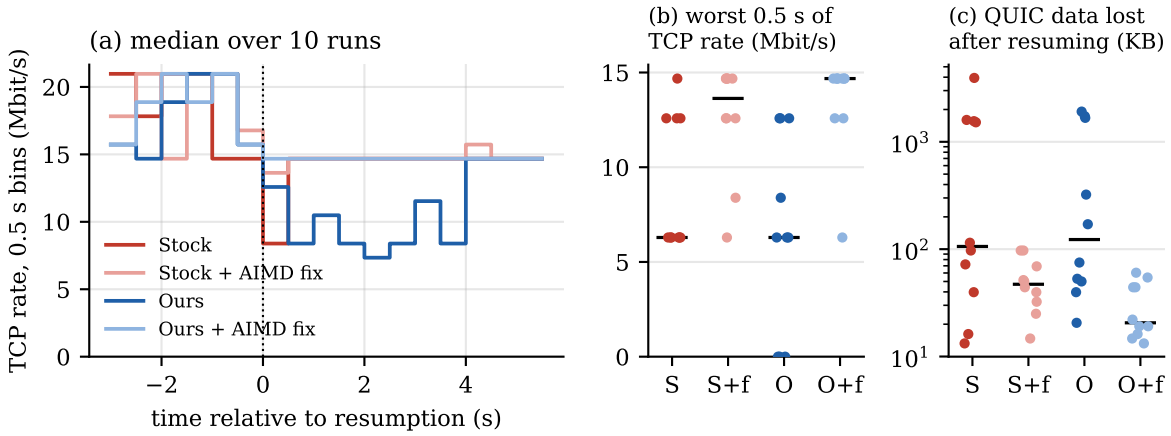


Figure 12: Competing TCP flow (25 Mbit/s). (a) TCP rate around resumption, median of 10 runs; (b) worst 0.5 s of TCP rate in the 2 s after resumption; (c) QUIC data lost after resuming. S = stock, O = ours, +f = with AIMD fix.

8 Unit tests

All 57 tests of MsQuic's `CubicTest` suite pass with the patches. Table 8 lists the tests that were added or changed. Each new test fails without the change it covers.

Table 8: New and updated unit tests in `src/core/unittest/CubicTest.cpp`.

Test	Change	What it checks
`RateLimited_MaxFsResetOnLoss`	C1, C2	After losing 2400 B out of 12 320 B in flight, *maxFS* equals the remaining 9920 B, not the old peak.
`RateLimited_MaxFsResetOnEcn`	C1, C2	After an ECN mark, *maxFS* equals the bytes still in flight.
`RateLimited_SpuriousCongestion-RestoresMaxFs`	C5	A spurious-loss undo restores *maxFS* to its pre-loss value.
`RateLimited_CongestionAvoidance-CapIsMaxFsPlusSmss`	C3	After 30 ACKs in congestion avoidance with a constant flight, $cwnd = maxFS + 1232$.
`SlowStart_WindowOverflowAfter-PersistentCongestion` (updated)	C3	Previously asserted the old $2 \cdot maxFS$ limit; now asserts $cwnd = maxFS + SMSS$.
`HyStart_ConservativeSlowStart-Rounds_TransitionToDone` (updated)	C3	Asserts $cwnd \le ssthresh$ and $cwnd \le 2 \cdot maxFS$ in slow start.
`AIMD_AccumulatorNoUnderflow`	fix	The accumulator crosses `AimdWindow` by 50 B; it must keep the 50 B remainder and a further small ACK must not grow the window again.
`AIMD_-AccumulatorTriggersWindowGrowth`, `AIMD_SequentialLinearConvergence` (updated)	fix	Expected values follow the corrected order (subtract, then grow).

9 Discussion and limitations

- **Ship the bug fix with the patch.** The draft's cap $maxFS + SMSS$ bounds growth *per measured flight*; on a real network the flight grows with every ACK, so a controller that grows by one packet per ACK (the accumulator bug) passes through it. Our patch without the fix was the worst build under competition (Section 7.3).

- **Conservative start on high-BDP paths.** When the resumed burst overflows a shallow queue anyway, a smaller starting window leads to a smaller window after the first loss, which CUBIC grows back slowly. At 50 Mbit/s with a 100-packet queue this cost 23 % in completion time of an 8 MB transfer. Pacing the resumed burst would help both builds.
- **Loss, not delay.** With a real 100-packet queue the queueing delay is bounded by the queue, so the harm of the stock burst shows as loss (up to 3.8 MB lost and up to ten times the bottleneck drops) rather than as extra latency for other flows. A longer queue (bufferbloat) would turn it into delay, as our exploratory GSO-enabled run showed.
- **Oscillation.** Without lowering *ssthresh*, a rate-limited sender's window can move between $maxFS + SMSS$ and $\min(ssthresh, 2 \cdot maxFS)$ (Scenario B). This is bounded by validated values and was the better trade-off (Section 4.1).
- **Initial** *maxFS***.** The draft initialises *maxFS* to the initial window; MsQuic initialises it to half of it, and we kept that. It only matters before the first window reduction, when it makes the slow-start cap $2 \cdot maxFS$ equal to the initial window until the flight exceeds half of it. Every result here is measured after a reduction, where the draft's reset applies.
- **Scope.** Only CUBIC is changed; MsQuic's BBR has its own application-limited handling. The cap uses the datagram payload size of path 0, like the rest of MsQuic's CUBIC, so it is not multipath-aware.
- **Testbed.** One emulated path with a drop-tail queue, one application pattern, at most one competing flow, 10 runs per point. The competing-TCP results are noisier than the others (bimodal outcomes), so differences there have wider intervals.
- **Scenario harness.** The scenarios use idealised whole-round ACKs. They isolate the logic of each change; the testbed covers network behaviour.

10 Conclusion

A rate-limited sender should not be allowed to claim capacity it never used. Stock MsQuic CUBIC already tried to enforce this, but its limit was based on a stale maximum and was twice as loose as the draft allows in congestion avoidance. With five small changes (34 lines of code) we implement the rules of `draft-ietf-ccwg-ratelimited-increase-11`, and the ablation shows where each change matters: C1–C3 stop the resumption burst (76 → 29 packets), C4 keeps the window legal for tiny flights, C5 keeps spurious-loss undo working, and leaving *ssthresh* alone keeps ramp-up fast.

The testbed evaluation also exposed an upstream bug, the underflow of CUBIC's Reno-friendly accumulator, which inflated stock MsQuic's window and which our change cannot contain on its own; we fixed it. With the fix in both builds, our change removes the burst after long rate-limited periods (*cwnd* 670 → 166 KB, data lost 1094 → 32 KB after 28 s, $p < 0.001$), reduces losses by 34–63 % after shorter ones and, together with the fix, keeps a competing TCP flow's worst half-second at 14.7 instead of 6.3 Mbit/s with stock MsQuic. It costs no transfer time except on a high-BDP path with a shallow queue.

References

[1] M. Welzl, T. Henderson, G. Fairhurst, M. P. Tahiliani, *Increase of the Congestion Window when the Sender Is Rate-Limited*, IETF Internet-Draft `draft-ietf-ccwg-ratelimited-increase-11`, September 2026.

[2] J. Iyengar, I. Swett, *QUIC Loss Detection and Congestion Control*, RFC 9002, 2021.

[3] L. Xu, S. Ha, I. Rhee, V. Goel, L. Eggert, *CUBIC for Fast and Long-Distance Networks*, RFC 9438, 2023.

[4] G. Fairhurst, A. Sathiaseelan, R. Secchi, *Updating TCP to Support Rate-Limited Traffic*, RFC 7661, 2015.

[5] M. Allman, V. Paxson, E. Blanton, *TCP Congestion Control*, RFC 5681, 2009.

[6] Microsoft, *MsQuic*, `https://github.com/microsoft/msquic`.

A Audit of the earlier plots in `experiments/`

Before this report, a number of exploratory plots had been made in `experiments/`. We checked each against the raw data and the code. None of them is used in this report. Every figure here was regenerated from the raw traces by three scripts:

- `report/make_figures.py`,
- `experiments/analyze_campaign.py`,
- `experiments/analyze_v3.py`.

The old plots are kept in `experiments/old_plots/`. Corrected versions of the same ideas are drawn into `experiments/graphs/` by `experiments/make_experiment_graphs.py`.

Table 9: Verdict on each earlier plot in `experiments/`.

Plot(s)	Verdict	Reason
`gap1_evidence_clean(_new)`, `gap1_harm_evidence`	correct	Show the stale *maxFS* correctly, but not its consequence; superseded by `graphs/gap1_before_after.png`.
`resumption_bomb*`, `bomb_attempt`	invalid	Client and server logs interleaved in one series, producing vertical spikes; titles claim a burst the plotted data does not show.
`before_cap_comparison*`, `gap1/gap2_after`	invalid	Caps computed from the plot title instead of the data, and from an earlier "+1 byte" version of the cap; `gap1_after` is from a superseded build.
`appendix_*`, `worked_example`, `rls_problem`	incomplete	Slow-start only (miss the CA rule) and mix bytes and packets on one axis; one has a 175 s axis artefact.

Conventions in the new figures. One colour per build throughout (stock red, patched blue; lighter shades for the builds with the AIMD fix); shared axes where panels are compared; units on every axis; the scenario figures in packets of 1232 B, the testbed figures in KB; every network figure states the run it shows or how many runs it summarises; all figures are vector PDFs.

B Reproducing the results

```
# apply the patches to upstream MsQuic 464c873c0
cd msquic && git am ../patches/*.patch
# unit tests
cmake --build build-test --target msquiccoretest
build-test/bin/Debug/msquiccoretest --gtest_filter='CubicTest.*'
# ablation (rebuilds cubic.c seven times, restores sources on exit)
report/ablation/run_ablation.sh
# harness: accumulator bug (scenario F), stock and stock + fix
report/ablation/run_f.sh stock > report/ablation/f_stock.txt
```

```
SRC=$PWD/report/ablation/cubic_stock_aimdfix.c \
  report/ablation/run_f.sh stockfix > report/ablation/f_stockfix.txt
# testbed campaigns (need root for network namespaces)
sudo experiments/run_campaign.sh        # sweep, 220 runs
sudo experiments/run_campaign.sh fix    # four builds, 200 runs
python3 experiments/analyze_campaign.py
python3 experiments/analyze_v3.py
python3 experiments/make_experiment_graphs.py   # experiments/graphs/
# figures and this document
python3 report/make_figures.py
python3 report/make_examples.py      # examples, bug_harness, scenario_e
cd report && tectonic report.tex
```